

Qué es la solución?

La solución es una plataforma de tipo Backend as a Service, pensada para equipos de desarrollo que construyen sistemas donde múltiples usuarios o procesos interactúan al mismo tiempo sobre datos compartidos. Esto incluye desde startups hasta empresas que desarrollan productos como e-commerce, sistemas de reservas o herramientas colaborativas.

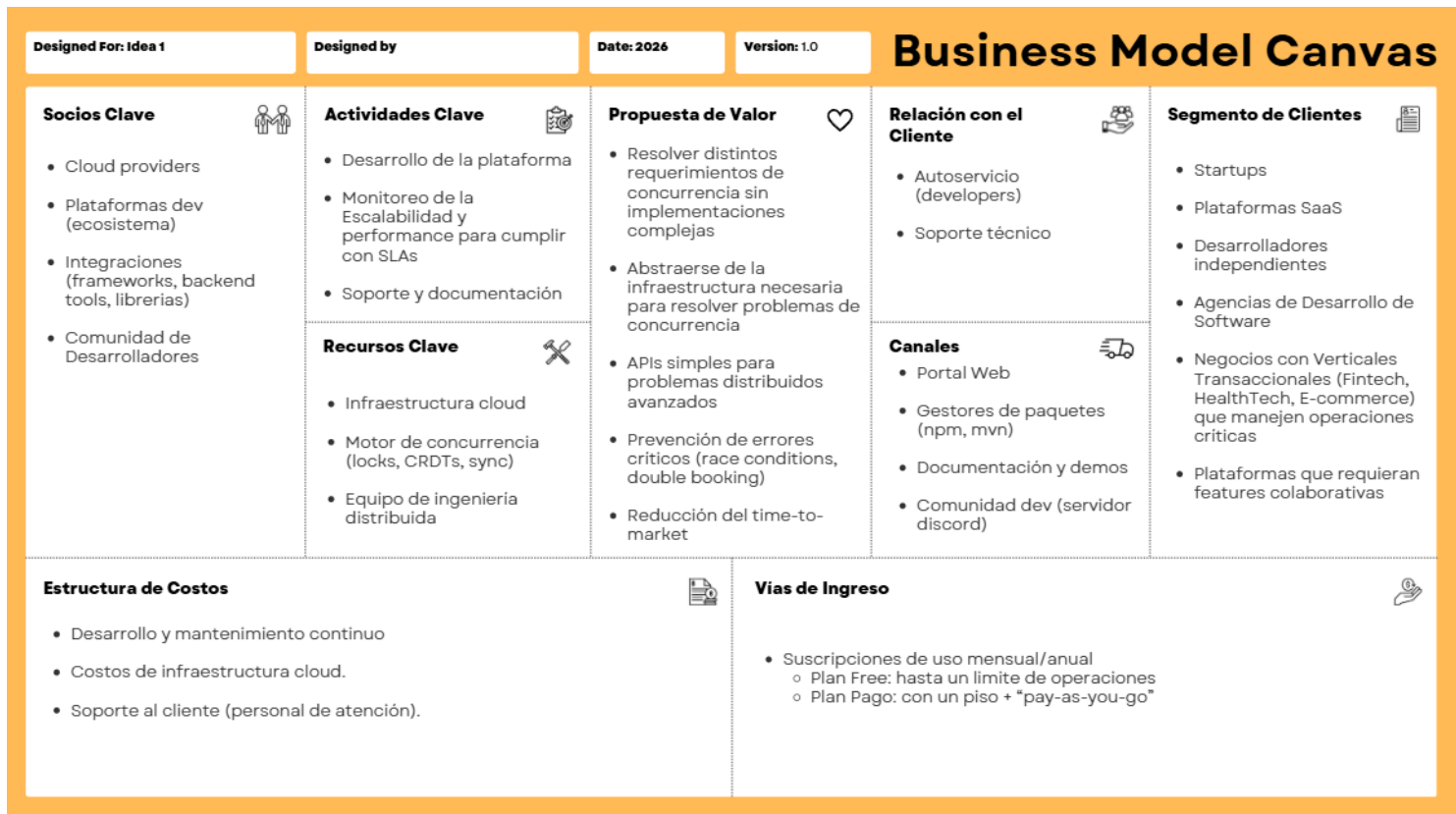
Se usaría en situaciones donde aparecen problemas típicos de concurrencia, por ejemplo:

- múltiples usuarios intentando acceder o reservar un mismo recurso (asientos, stock, turnos)
- procesos distribuidos que necesitan coordinarse para evitar inconsistencias

En todos estos casos, los desarrolladores suelen tener que implementar soluciones complejas: manejo de locks, control de consistencia, resolución de conflictos, sincronización en tiempo real, entre otros.

La plataforma propone resolver estos problemas como un servicio, permitiendo que el desarrollador simplemente configure el comportamiento deseado e integre APIs simples, en lugar de implementar lógica distribuida desde cero, abstrayéndose así de la complejidad y enfocándose en el producto.

Business Model Canvas



Flujo de Uso General

1. Registro e inicio de sesión: El desarrollador crea una cuenta en la plataforma e ingresa al dashboard.
2. Creación de una aplicación:
3. Define una nueva aplicación donde agrupa sus integraciones (por entorno: dev, staging, prod) (*esto es más una idea, no algo 100% definido*).
4. Configuración de un componente: Selecciona el tipo de componente que necesita (gestión de recursos, locks distribuidos o colaboración) y configura sus parámetros según el caso de uso.
5. Obtención de credenciales: La plataforma genera API keys y datos de acceso para integrar el servicio.
6. Integración mediante SDK o API: El desarrollador incorpora el SDK o utiliza los endpoints provistos, siguiendo ejemplos y snippets de código.
7. Uso desde su sistema: Su aplicación comienza a interactuar con la plataforma (reservas, locks, sincronización, etc.), delegando la lógica de concurrencia.
8. Monitoreo y ajustes: Desde el dashboard puede revisar el uso y ajustar configuraciones según necesidad. (*tampoco hay algo definido respecto a esto*)

Este flujo permite pasar de la configuración a una integración funcional de forma rápida, sin necesidad de implementar lógica compleja desde cero.

Componentes de la solución

Shared Resource Engine

Es un componente que permite gestionar el acceso concurrente a recursos compartidos mediante un modelo de reserva temporal + confirmación, evitando conflictos como doble asignación, sobreventa o falta de liberación.

Configuración propuesta

- Nombre del recurso: identificación interna (ej: seat, trip_capacity, appointment_slot)
- Descripción
- Environment: separación entre entornos (dev, staging, prod).
- Modo de recurso
 - Unitario: cada recurso es una instancia individual.
 - Múltiple: un recurso representa una cantidad (stock/cupos).
- Guardar metadata (si/no)
- TTL por defecto: Tiempo máximo que una reserva permanece activa antes de liberarse automáticamente.
- Estrategia ante conflicto
 - Fail: error inmediato si no hay disponibilidad.
 - Retry: reintentos automáticos.
 - Queue: encolado hasta disponibilidad.
- Configuración de retry (si aplica): Intervalo y cantidad máxima de reintentos.
- Habilitar idempotencia: Permite evitar efectos duplicados ante reintentos.
 - Ubicación de la clave: Define dónde se envía (header o body).
 - TTL de idempotencia: Tiempo durante el cual se recuerda una operación.
- Webhook: url de webhook para enviar actualizaciones

Operaciones básicas

- lock: bloquea temporalmente
- confirm: confirma definitivamente
- release: libera manualmente
- availability: consulta disponibilidad de uno o más recursos

Ejemplo de uso

Escenario

El Escenario: Una cadena de cines lanza la venta para el estreno de una película. Dos usuarios ven el Asiento "J4" libre e intentan comprarlo exactamente al mismo milisegundo. Se necesita retener el asiento 5 minutos mientras pagan.

Cómo se resolvería actualmente sin nuestra solución (asumiendo una base de datos relacional SQL):

1. Definir en la tabla de la base de datos "Butacas" una columna estado con posibles valores (DISPONIBLE, RESERVADO, PAGA) donde reservado es el estado intermedio donde la persona tiene 5 min para poder hacer efectiva su reserva. Además se deberá definir otra columna, bloqueado_hasta para saber hasta cuando esa butaca aparecerá como bloqueada para otros usuarios
2. En cada caso que se quiera reservar una butaca, se deberá iniciar una transacción de bases para las butacas marcadas como DISPONIBLE, cambiando su estado a RESERVADA en la misma transacción, dejando afuera a otros usuarios. Si para el usuario "perdedor" la request retorna 0 columnas al clickear, se deberá traducir ese resultado vacío a un mensaje de error de "lo sentimos butaca no disponible"
3. Si la persona que logró entrar en la ventana de 5 min paga la butaca, se cambia el estado a PAGADA y si cancela vuelve a DISPONIBLE. Pero hay un caso borde más el que es si el usuario deja la transacción por la mitad. Ahí para eso debería setearse un timer o scheduler que vuelva a poner a la butaca en disponible de forma automática, por ejemplo usando el campo "bloqueado_hasta" definido anteriormente

Cómo se resolvería con nuestra solución:

- Recurso: **seat** (unitario)
 - Cada recurso representa una butaca en una función específica
1. (Configuración): En el panel de la plataforma, el cliente define que el recurso "asiento_cine" tiene un TTL por defecto de 5 minutos y estrategia de conflicto "Fail" (si no está, falla rápido).
 2. (La Reserva): El backend del cliente simplemente llama a la API expuesta: `SharedResource.reserve('asiento_J4_funcion_{id}')`. La plataforma maneja toda la concurrencia en memoria RAM ultrarrápida (ej. Redis).
 3. Si la reserva es confirmada, se llama a `SharedResource.confirm('asiento_J4_funcion_{id}')` apagando el TTL del recurso compartido y confirmandolo, en cambio si el usuario se arrepiente cancelando su reserva activamente, se llama `SharedResource.release('asiento_J4_funcion_{id}')`
 4. (Manejo de abandono): El Usuario 1 se va sin pagar. El cliente no hace nada.
 5. (Liberación automática): A los 5 minutos exactos, la plataforma expira la reserva internamente y opcionalmente le dispara un Webhook al cliente (`reservation.expired`) para que actualice la UI de su frontend.

Beneficios

- Abstracción de lógica compleja de concurrencia
- Reducción de errores críticos
- Configuración declarativa en lugar de implementación manual
- Soporte para múltiples casos de uso con un mismo modelo
- Escalabilidad automática
- Menor tiempo de desarrollo

Distributed Lock Service

Permite garantizar exclusión mutua sobre recursos en entornos distribuidos, evitando que múltiples procesos ejecuten simultáneamente operaciones críticas sobre una misma entidad.

A diferencia del Shared Resource Engine (orientado a negocio), este componente es más bajo nivel y genérico, pensado para resolver problemas de sincronización entre procesos o servicios.

Configuración propuesta

- Nombre del recurso: identificación interna (ej: seat, trip_capacity, appointment_slot)
- Descripción
- Environment: separación entre entornos (dev, staging, prod)
- Tipo de lock
 - Exclusive: solo un cliente puede adquirirlo.
 - Read/Write: múltiples lectores simultáneos, un único escritor exclusivo
- TTL del lock: Tiempo máximo que un lock puede mantenerse activo. Evita deadlocks en caso de fallos del cliente.
- Estrategia de adquisición
 - Fail: si el lock está ocupado, devuelve error.
 - Retry: reintenta automáticamente hasta adquirirlo.
 - Blocking: espera hasta que el lock se libere.
- Configuración de retry (si aplica)
 - Intervalo entre intentos
 - Máximo de reintentos
- Fencing tokens (opcional pero recomendado): Cada adquisición devuelve un token incremental único. Permite evitar problemas de “clientes viejos” que siguen operando tras perder el lock.
- Nivel de consistencia: Define garantías del sistema subyacente (*a definir*)
 - Strong: asegura exclusión estricta
 - Eventual: mayor performance, menor garantía
- Namespace / scope. Permite agrupar locks por aplicación o contexto. Evita colisiones entre distintos sistemas (*también, es solo una idea*)
- Webhook: url de webhook para enviar actualizaciones

Operaciones Básicas

- acquire lock: Solicita un lock sobre una clave (ej: `order_123`)
- release lock: Libera el lock manualmente
- renew lock: Extiende el TTL si la operación sigue en curso

Ejemplo de Uso

Escenario

Un sistema de microservicios necesita cobrar la suscripción mensual de un usuario. Debemos garantizar que una pausa del sistema (GC pause) no provoque que dos workers distintos le cobren dos veces a la misma tarjeta.

Cómo se resolvería actualmente sin nuestra solución (asumiendo una base de datos relacional SQL):

1. El equipo no puede usar una base de datos normal para el lock. Tienen que levantar un clúster de al menos 3 nodos de Apache ZooKeeper (para garantizar consenso y evitar el Split-Brain). Esto requiere configurar servidores, redes, parches de seguridad y monitoreo constante.

2. El código de adquisición

El desarrollador tiene que importar librerías complejas (como Apache Curator en Java). Escribe el código para pedirle un lock a ZooKeeper y extraer el zxid (el número de versión estrictamente incremental) para usarlo como Fencing Token.

3. Modificar la Base de Datos de Pagos

El equipo modifica la tabla suscripciones en su base de datos PostgreSQL añadiendo una columna: `ultimo_token_procesado`.

4. El escenario de fallo (GC Pause).

El Worker A pide el lock a ZooKeeper. ZooKeeper se lo da y le asigna el Token #33.

El Worker A sufre una pausa del Garbage Collector y se congela. El lock en ZooKeeper expira por timeout.

El Worker B pide el lock. ZooKeeper se lo da con el Token #34.

El Worker B envía a PostgreSQL: "Cobrar \$10 y guardar Token #34". PostgreSQL cobra y anota que último token es el 34.

5. La salvación manual.

El Worker A "despierta" de su pausa. Como no sabe que estuvo dormido, intenta cobrar enviando: "Cobrar \$10 y guardar Token #33".

PostgreSQL compara: ¿33 es mayor que 34? No. La base de datos rechaza la transacción. Se evitó el doble cobro.

Cómo se resolvería con nuestra solución: (también basada en fencing tokens)

El equipo no configura servidores, ni nodos, ni algoritmos de consenso. Solo crean una cuenta en tu plataforma y obtienen una API Key.

Adquisición en una línea de código.

El desarrollador usa el SDK. La plataforma (que tiene el consenso fuerte resuelto internamente) genera el token matemáticamente seguro:

```
// El SDK abstrae toda la complejidad distribuida
const lock = await DistributedLock.acquire('pago_user_123', {
  ttl: '10s',
  requireFencingToken: true
});
// lock.fencingToken ahora vale 33
```

Modificar la Base de Datos (Igual que antes).

Nota importante: Como señala [Kleppmann](#), el almacenamiento final (PostgreSQL) debe participar activamente. El cliente sigue teniendo que agregar la columna ultimo_token en su base de datos para rechazar tokens viejos. Eso es inevitable por leyes de la física de sistemas.

El escenario de la pausa ocurre exactamente igual. El Worker A se duerme, el Worker B usa el Token #34 de tu plataforma. Cuando el Worker A despierta e intenta usar el Token #33, la base de datos lo rechaza.

El resultado es el mismo éxito transaccional, pero el cliente se ahorró ingeniería e infraestructura.

Resultado:

- Solo un proceso ejecuta el cobro
- El resto falla o espera

Beneficios

- API simple para locks distribuidos
- Abstracción de herramientas complejas (ZooKeeper, Redis, etc.)
- Prevención de errores críticos de concurrencia
- Configuración flexible según necesidad
- Manejo automático de TTL, retries y fallos
- Soporte para patrones avanzados (read/write, fencing tokens)